VELOOP Rewards – Giveaway Section Frontend Development Task

THE DESIGN IMAGES PROVIDED USE IT AS YOU REFERENCE DON'T CLONE IT, & THE ASSETS ALSO PROVIDED…..

ASSETS:
https://drive.google.com/drive/folders/1CgffcCUUo89deG8xR8grzmy4rEidNPl_?usp=sharing

REFERENCE:
https://drive.google.com/drive/folders/1G3LIdcgKUJmy9mPcD0QuMDcj3vsT4hyi?usp=sharing

Project: Complete Giveaway Page + Giveaway Banner/Card

Task Type

Frontend Development

Objective

Develop a completely polished **Giveaway Section** for VELOOP Rewards, including:

1. Giveaway Page
2. Giveaway Banner/Card
3. Giveaway participation experience
4. Giveaway status handling
5. Winners section
6. Previous Winners section
7. Winner-specific claim/details form
8. Winner announcement slider/messages
9. Responsive design
10. Interactive states and animations

The final experience should make users feel:

"This is a real, trustworthy reward opportunity, and I want to participate."

The UI should be **premium, realistic, rewarding, engaging and interactive** without looking cheap, overly flashy, or like a casino/gambling website.

---

1. Reference Designs

Two reference images have been provided with this task.

Use the provided images as the **visual direction/reference** for:

- Overall giveaway page composition
- Giveaway banner
- Typography hierarchy
- Reward presentation
- Giveaway cards
- Winner presentation
- Premium reward atmosphere
- Gift/reward illustrations
- Layout inspiration

Important

Do not simply copy the images.

The images are references for the desired quality and concept.

You are expected to create a **VELOOP-branded implementation with your own UX improvements and creative ideas.**

---

2. Existing Assets

The provided images/assets should be used where appropriate.

You may also use additional suitable:

- Icons
- Illustrations
- SVGs
- React icon libraries
- Supporting visual assets

However, do not overload the interface with unnecessary graphics.

Every visual element should have a purpose.

---

3. Main Design Direction

The giveaway experience should feel:

- Premium
- Modern

- Reward-focused
- Trustworthy
- Interactive
- Exciting
- Realistic
- Engaging
- Branded
- Fintech-inspired

The page should communicate:

Excitement

"Something valuable could be yours."

Trust

"This giveaway is transparent and legitimate."

Participation

"I should join before it ends."

Social proof

"Other users are actually winning."

---

4. Avoid Cheap/Flashy Design

Do NOT create:

- Excessive neon
- Too many gradients
- Flashing animations
- Casino-style visuals
- Excessive confetti
- Overly saturated colors
- Random glowing effects
- Too many floating objects
- Cheap-looking reward cards

Use animation and glow effects **intentionally and subtly**.

---

5. Giveaway Banner/Card

Create a large premium giveaway banner similar in visual importance to the provided reference.

The banner should include:

Giveaway Label

Example:

□ EXCLUSIVE GIVEAWAY

Main Heading

Example:

Giveaway Rewards

or a more creative heading.

Supporting Text

Example:

Complete eligible activities, collect entries and get a chance to win exciting rewards.

CTA

Enter Giveaway →

or:

Join Giveaway →

---

6. Giveaway Hero Illustration

The banner must have a strong visual illustration.

Possible elements:

- Premium gift box
- Reward ticket
- Smartphone

- Coins
- Gift cards
- Trophy
- Shopping rewards
- Floating reward elements

The illustration should immediately communicate:

This is a giveaway/reward section.

---

7. Banner Interaction

The banner should not feel like a static image.

Add subtle interactions such as:

- Hover animation
- Illustration movement
- CTA animation
- Soft background motion
- Reward glow
- Floating particles
- Subtle parallax

Keep these effects restrained.

---

8. Giveaway Page Hero

The complete page should have a strong hero section.

Example:

EXCLUSIVE GIVEAWAYS

Giveaway Rewards

Complete tasks. Earn entries.
Get a chance to win.

[Join Now →]

You can create a significantly better layout.

---

9. Giveaway Statistics

Include a statistics section.

Possible statistics:

Total Giveaways

Example:

24 Active

Total Participants

Example:

8.5K+ Users

Prizes Won

Example:

1.2K+ Rewards

Giveaway Ends In

Example:

12d : 08h : 45m

Use **dummy data for frontend development**.

---

10. Countdown Timer

If the giveaway is active, show a live countdown.

Example:

Ends In

12d : 08h : 45m : 32s

The timer should update automatically.

When the countdown reaches zero, the UI must automatically switch to the ended state.

---

11. Giveaway Status

Support at least three states:

ACTIVE

Giveaway Live
Ends in 4d 12h
[Join Giveaway]

ENDED

Giveaway Ended

Winner announcement is now available.
[View Winners]

UPCOMING

Next Giveaway Starts In

3 Days

Get ready for the next reward.

---

12. Giveaway Cards

Create attractive giveaway/reward cards.

Example prizes:

1st Prize

iPhone 15 Pro

2nd Prize

Apple Watch

3rd Prize

AirPods

Lucky Draw

Amazon Gift Card

Use the assets provided with the task where appropriate.

---

13. Prize Card Information

Every prize card should contain:

- Prize position
- Prize image
- Prize name
- Short description
- Number of participants
- Remaining time
- Entry requirement if applicable
- CTA

Example:

1st Prize

iPhone 15 Pro

2.3K+ Participants

12d remaining

[Join Now →]

---

14. Giveaway Participation

Create a clear participation flow.

A user should understand:

Step 1

Sign up / Login

Step 2

Complete eligible activities

Step 3

Earn entries

Step 4

Participate in the giveaway

Step 5

Winner is selected after the giveaway ends

---

15. How to Participate Section

Create a dedicated section:

How to Participate?

Use a visual step-by-step timeline.

Example:

01
Sign Up / Login
↓
02
Complete Tasks
↓
03
Earn Entries
↓
04
Win Rewards

Use meaningful icons/illustrations.

---

16. Entry System

The frontend should visually support an entry-based giveaway system.

Example:

Your Entries

24 Entries

or:

Complete eligible tasks to earn more entries.

If actual entry functionality is not available, use dummy data.

---

17. Participation CTA

The primary CTA should change based on the user's state.

Not Logged In

Login to Participate

Logged In – Not Joined

Join Giveaway

Already Joined

You're Participating ✓

Giveaway Ended

View Winners

Upcoming

Notify Me

Use frontend state for demonstration.

---

18. Winner Announcement Slider

Important Requirement

Create a horizontal **winner announcement/message slider**.

This should create social proof.

Example messages:

 A user just won an **iPhone 15 Pro**

 Another lucky participant won an **Apple Watch**

 A participant won **AirPods Pro**

 Someone won an **Amazon Gift Card**

---

19. Winner Slider Rules

The slider should:

- Automatically rotate
- Have smooth transitions
- Pause on hover where appropriate
- Support mobile screens
- Not move too quickly
- Be readable

Use dummy names/user IDs.

Do not expose personal information.

Example:

 User VE****72 won an iPhone 15 Pro

---

20. Winner Distribution

For the current giveaway, demonstrate:

One iPhone Winner

There should be **only one winner for the iPhone/1st prize**.

Multiple Winners

Other prizes can have multiple winners depending on the giveaway configuration.

Example:

iPhone 15 Pro
1 Winner

Apple Watch
3 Winners

AirPods Pro
5 Winners

Amazon Gift Card
10 Winners

These numbers are for frontend demonstration and should be configurable.

---

21. Winners Tab

Create a dedicated:

Winners

tab.

When the current giveaway is active:

The section should communicate that winners have **not yet been finalized**.

Example:

Giveaway is still live

Winners will be announced after the giveaway ends.

Show:

Current Giveaway
Status: LIVE

Winner announcement:
Coming after the giveaway ends.

Do not falsely display winners for an active giveaway.

---

22. Previous Winners Tab

Create another tab:

Previous Winners

This should contain winners from completed giveaways.

Example:

Previous Giveaway

August Reward Rush

Winner:
VE****82

Prize:
iPhone 15 Pro

Date:
05 Aug 2026

---

23. Giveaway Lifecycle

The page must support the following lifecycle.

Stage 1 – Current Giveaway

ACTIVE

Current Giveaway
↓
Users participate
↓
Countdown running
↓
No winner yet

Stage 2 – Giveaway Ends

COUNTDOWN = 0

↓

Giveaway Ended
↓
Winner selection state
↓
Winners displayed

Stage 3 – Next Giveaway

After the current giveaway ends:

Next Giveaway Starts In

3 Days

The page should display this state.

Stage 4 – New Giveaway Starts

When the next giveaway begins:

New Giveaway
ACTIVE

Previous winners
↓
Previous Winners tab

The newly selected winners belong to the current giveaway lifecycle.

---

24. Winner History

Previous winners should show:

- User display name/masked ID
- Prize
- Giveaway name
- Date
- Prize category
- Winner status

Example:

VE****42

Won:
Apple Watch Series 9

Giveaway:
Summer Rewards

Date:
06 Aug 2026

---

25. Winner Privacy

Do not display:

- Full email
- Phone number
- Full address
- Sensitive personal information

Use:

VE****42

or a safe display name.

---

26. Winner-Specific Claim Area

Critical Requirement

If the logged-in user's ID matches the winner list, the page must show a special winner section.

Example:

🎉 Congratulations!

You won **Apple Watch Series 9**!

Then show:

Claim Your Prize

---

27. Winner Verification

Before allowing prize details submission, show:

- Prize
- Giveaway name
- Winner status
- Claim deadline
- Required information

Example:

Prize:
Apple Watch Series 9

Status:
Winner ✓

Claim within:
7 days

---

28. Physical Prize Form

For physical prizes such as:

- iPhone
- Apple Watch
- AirPods
- Other physical products

collect:

Full Name

Phone Number

Complete Address

City

State

PIN Code

Only collect information required for prize fulfillment.

29. Amazon Gift Card Form

For Amazon Gift Card winners, do **NOT** ask for a physical delivery address.

Instead collect:

Email Address

Example:

Enter email where you want to receive your gift card.

---

30. Prize Claim Modal

The form can open in a premium modal.

Example:

```
┌──────────────────────────────────┐
| 🎁 Claim Your Prize              |
|                                  |
| You won Apple Watch Series 9     |
|                                  |
| Full Name                        |
| [_____________________________]  |
|                                  |
| Phone Number                     |
| [_____________________________]  |
|                                  |
| Address                          |
| [_____________________________]  |
|                                  |
| City      State      PIN         |
|                                  |
| [ Submit Claim ]                 |
└──────────────────────────────────┘
```

---

31. Amazon Gift Card Modal

```
┌──────────────────────────────────┐
| 🎁 Claim Gift Card               |
|                                  |
| You won ₹2,000 Amazon Gift Card  |
|                                  |
| Email Address                    |
| [_____________________________]  |
|                                  |
| [ Submit Claim ]                 |
└──────────────────────────────────┘
```

32. Winner Claim States

Support:

Not Submitted

Claim Prize

Submitted

Claim Submitted ✓

Our team will process your prize.

Processing

Prize Verification
In Progress

Completed

Prize Delivered ✓

Expired

Claim Window Expired

These can be simulated using frontend dummy states.

---

33. Non-Winner Experience

Users who did not win should **not** see the winner claim form.

They should only see:

- Winner list
- Previous winners
- Giveaway status
- Next giveaway
- Participation CTA

Example:

Didn't win this time?

Keep participating for the next giveaway.

[Explore Next Giveaway →]

---

34. Current Winner Matching

For frontend development, create dummy logged-in user state.

Example:

currentUserId = "VE10025"

Winner list:

VE10025 → Apple Watch

The UI should detect:

```
currentUserId === winner.userId
```

and display the claim option.

35. Important Backend Note

Although this task is frontend-only, structure the frontend so that later it can consume:

```
GET /giveaways/current
GET /giveaways/:id
GET /giveaways/:id/winners
GET /giveaways/previous
GET /giveaways/my-status
POST /giveaways/:id/join
POST /giveaways/:id/claim
```

Do not tightly couple the UI to hardcoded arrays.

---

36. Giveaway Data Structure

Create mock data that resembles a real API response.

Example:

```
{
  id: "GW-2026-08",
  title: "Summer Rewards Giveaway",
  status: "active",
  startDate: "...",
  endDate: "...",
  prizes: [],
  participants: 8500
}
```

This will make future backend integration easier.

---

37. Prize Configuration

Prize data should contain:

- ID

- Name
- Position
- Image
- Description
- Winner count
- Prize type
- Claim type

Example:

```
{
  id: "PRIZE-001",
  name: "iPhone 15 Pro",
  type: "physical",
  winnerCount: 1
}
```

---

38. Prize Types

Support at least:

```
PHYSICAL
GIFT_CARD
DIGITAL
```

This allows the correct claim form to be displayed.

---

39. Responsive Requirements

The entire page must work across:

Mobile

320px+

Tablet

768px+

Laptop

1024px+

Desktop

1440px+

Large Screens

1920px+

40. Mobile Design

Do not simply shrink the desktop layout.

On mobile:

- Hero should stack
- Prize cards can become horizontal scroll/carousel
- Statistics should become 2-column or stacked
- Winner slider should remain readable
- Tabs should be touch-friendly
- Claim form should fit smaller screens
- CTA should be easily accessible

---

41. Tablet Design

Use a balanced grid.

For example:

Prize 1 Prize 2
Prize 3 Prize 4

depending on available width.

---

42. Desktop Design

Use a premium composition such as:

Hero
↓
Stats
↓
Featured Giveaways + How It Works
↓
Winner Announcement
↓
Winners / Previous Winners

↓
Trust Section
↓
CTA
↓
Footer

You may create a better structure.

---

43. Trust Section

Add a dedicated trust section.

Possible points:

100% Transparent

Giveaway rules are clearly displayed.

Secure

User information is handled responsibly.

Fair Participation

Participation rules are clearly explained.

Reward Transparency

Prize information is clearly displayed.

Do not make unsupported claims such as "100% guaranteed" unless VELOOP officially guarantees them.

---

44. Giveaway Rules

Add:

Giveaway Rules & Guidelines

Include dummy/example rules such as:

- Eligibility

- Participation requirements
- Entry rules
- Winner selection
- Prize claim period
- Disqualification conditions
- Fraud/abuse policy

The intern should make this visually accessible.

---

45. FAQ

Add a compact FAQ section if appropriate.

Possible questions:

How do I participate?

How are winners selected?

When are winners announced?

What happens if I win?

How do I claim my prize?

Can I participate in multiple giveaways?

What happens after the giveaway ends?

Use expandable accordion interactions.

---

46. Giveaway State Animation

When the giveaway becomes active:

- Countdown appears
- CTA becomes active
- Status badge changes

When it ends:

- Countdown transitions to ended state
- Winners tab becomes active

- Claim options appear for winners

These can be simulated using frontend state and dummy timestamps.

47. Winner Reveal Animation

After the giveaway ends, create a premium winner reveal.

Possible:

```
Giveaway Ended
      ↓
Selecting Winners...
      ↓
Winner Revealed
      ↓
 iPhone Winner
```

Do not make it look like gambling.

Keep it polished and controlled.

---

48. Interactive Reward Cards

Prize cards should respond to:

- Hover
- Focus
- Click
- Join
- View details

Use subtle:

- Elevation
- Border transitions
- Image movement
- Shadow changes

---

49. Icons

You may use:

- React Icons
- Lucide React
- Bootstrap Icons
- Other suitable React icon libraries

Use consistent iconography.

---

50. Animation Libraries

You may use:

- Framer Motion
- CSS animations
- React animation libraries

Animations should improve UX rather than distract from content.

---

51. Technology Stack

Required

- React.js
- Vite
- Bootstrap
- CSS Modules (.module.css)
- React Hooks

Allowed

- React Icons
- Lucide React
- Framer Motion
- Other relevant React libraries

---

52. Component Architecture

Use reusable components.

Suggested:

```
src/
│
├── components/
│   ├── GiveawayHero/
│   ├── GiveawayStats/
│   ├── PrizeCard/
│   ├── FeaturedGiveaways/
│   ├── HowToParticipate/
│   ├── WinnerSlider/
│   ├── WinnersTabs/
│   ├── WinnerCard/
│   ├── PreviousWinnerCard/
│   ├── PrizeClaimModal/
│   ├── GiveawayRules/
│   ├── FAQ/
│   ├── TrustSection/
│   └── Countdown/
│
├── pages/
│   └── Giveaway/
│
├── data/
│   └── giveawayData.js
│
├── assets/
│
└── styles/
```

---

53. Dummy Data Requirement

Since this is a frontend task, use realistic dummy data.

For example:

Total Giveaways: 24
Participants: 8,500+
Prizes Won: 1,200+

Use realistic but clearly fictional development data.

Do not claim these are real VELOOP statistics.

---

54. User States

The frontend should support:

State 1

Visitor

State 2

Logged-in user not participating

State 3

Logged-in participant

State 4

Winner

State 5

Non-winner

State 6

Giveaway ended

State 7

Upcoming giveaway

---

55. Visitor State

Show:

Login / Signup to Participate

If they click participate:

Please login or create an account to participate.

---

56. Participant State

Show:

You're Participating ✓

Your Entries:
24

CTA:

Earn More Entries →

57. Winner State

Show:

 Congratulations!

You won:

Apple Watch Series 9

[Claim Your Prize]

---

58. Non-Winner State

Show:

Thanks for participating!

Winners have been announced.

[View Winners]

Better luck next time!

---

59. Upcoming State

Show:

Next Giveaway

Starts In

3 Days

Get ready for another chance to win.

[Explore Rewards]

---

60. Winner List Structure

Create a visually attractive winner list.

Example:

 Winners

VE****42
iPhone 15 Pro
August 10, 2026

VE****91
Apple Watch
August 10, 2026

VE****27
AirPods Pro
August 10, 2026

---

61. Previous Winner Transition

When a new giveaway begins:

Current Giveaway
↓
Giveaway Ends
↓
Winners announced
↓
Current winners move to
Previous Winners
↓
New giveaway becomes Current

The frontend should be designed to support this lifecycle.

---

62. Winner Slider Examples

Use messages such as:

 VE****21 won an iPhone 15 Pro!

 VE****83 won an Apple Watch!

 VE****54 won AirPods Pro!

 VE****92 won an Amazon Gift Card!

Use dummy user IDs.

---

63. Important – Don't Fake Real-Time Activity

If the frontend is using dummy data, don't present it as actual live activity.

For development, it can be labelled internally as mock/demo data.

When connected to the backend later, actual winner announcements can be displayed.

---

64. Empty States

Create proper empty states.

No Previous Winners

Previous winners will appear here after a giveaway is completed.

No Current Giveaway

The next giveaway is being prepared.

No Active Participation

Join the giveaway to start earning entries.

---

65. Loading States

Create skeleton/loading states for:

- Giveaway
- Prize cards
- Winners
- Previous winners
- Statistics
- Countdown

---

66. Error State

Example:

We couldn't load the giveaway information.

[Try Again]

Do not show raw JavaScript/API errors.

---

67. Accessibility

Make sure:

- Buttons are keyboard accessible
- Images have alt text
- Contrast is readable
- Modals can be closed
- Tabs have correct states
- Forms have labels
- Focus states are visible

---

68. Performance

Optimize:

- Large prize images
- Illustrations
- Animations
- Lazy-loaded assets

Avoid loading unnecessary heavy assets before they are needed.

---

69. No Hardcoded Business Logic

Keep giveaway configuration inside structured mock data.

Do not scatter:

```
if prize === "Amazon"
```

throughout multiple components.

Create reusable prize-type logic.

---

70. Local Development

Build and test the entire project on your **local system first**.

Required process:

1. Review the provided reference images.
2. Review the provided assets.
3. Plan the page architecture.
4. Create the giveaway data model.
5. Build the banner.
6. Build the giveaway page.
7. Build prize cards.
8. Build countdown.
9. Build winner slider.
10. Build winners tabs.
11. Build claim modal.
12. Build responsive layouts.
13. Test all user states.
14. Fix console errors.
15. Create production build.

---

71. GitHub Requirement

Push the complete project to GitHub.

The repository must contain:

- Source code
- Components
- Assets
- Mock data
- CSS Modules
- README.md
- Configuration files

Do not upload:

- node_modules
- .env
- Private credentials
- Unnecessary build files

---

72. README.md

Create a professional README containing:

- Project Overview
- Giveaway Concept
- Features
- Giveaway States
- Winner System
- Prize Claim System
- Technology Stack
- Installation
- Development
- Build
- Folder Structure
- Component Architecture
- Responsive Design
- Animation Details
- Mock Data Structure
- Future Backend Integration
- Screenshots
- Live Demo

---

73. Deployment

After completing the project locally, deploy it to:

- Vercel, or
- Netlify

Vercel is recommended.

Provide the live URL.

---

74. Final Deliverables

Submit:

1. GitHub Repository

Complete source code.

2. Live Website

Vercel/Netlify URL.

3. README.md

Complete documentation.

4. Screenshots

Provide screenshots of:

- Desktop
- Tablet
- Mobile
- Active giveaway
- Ended giveaway
- Upcoming giveaway
- Winner tab
- Previous winners
- Winner claim modal
- Amazon gift-card claim modal
- Non-winner state

---

75. Final Testing Checklist

Giveaway

- [ ] Hero/banner created
- [ ] Giveaway status
- [ ] Countdown
- [ ] Prize cards
- [ ] Participation CTA
- [ ] Entry display
- [ ] How-to-participate
- [ ] Rules
- [ ] FAQ
- [ ] Trust section

Winners

- [ ] Winner slider
- [ ] Winners tab
- [ ] Previous Winners tab
- [ ] Winner cards
- [ ] Masked user IDs
- [ ] One iPhone winner
- [ ] Multiple other prize winners
- [ ] Active giveaway doesn't falsely show current winners
- [ ] Ended giveaway shows winners
- [ ] New giveaway moves old winners to previous winners

Winner Claim

- [ ] Winner detection
- [ ] Claim button only for winner
- [ ] Physical prize form
- [ ] Name field
- [ ] Phone field
- [ ] Address field
- [ ] City
- [ ] State
- [ ] PIN
- [ ] Amazon gift card email form
- [ ] Submission state
- [ ] Processing state
- [ ] Completed state
- [ ] Expired state

Non-Winner

- [ ] Cannot access claim form
- [ ] Can see winners
- [ ] Can see previous winners
- [ ] Can participate in future giveaways

Technical

- [ ] React.js
- [ ] Vite
- [ ] Bootstrap
- [ ] CSS Modules
- [ ] React Hooks
- [ ] React Icons/Lucide
- [ ] Responsive
- [ ] Accessibility
- [ ] Animations
- [ ] Loading state
- [ ] Empty state
- [ ] Error state
- [ ] GitHub
- [ ] README
- [ ] Deployment

---

76. Quality Standard

The final result should look like a **real reward platform**, not a college/demo project.

When someone opens the page, the visual hierarchy should immediately communicate:

What is this?

→ Giveaway

What can I win?

→ Premium prizes

How do I participate?

→ Simple steps

Is this trustworthy?

→ Rules + transparency + secure experience

Are people actually winning?

→ Winner announcements + previous winners

Did I win?

→ Personalized winner state

What happens after I win?

→ Clear claim process

---

77. Final Product Experience

The intended user journey is:

Discover Giveaway
↓
See Rewards
↓
Understand Rules
↓
Join Giveaway
↓
Complete Eligible Activities
↓
Earn Entries
↓
Track Giveaway Countdown
↓
Giveaway Ends
↓
Winner Announcement
↓
Winner / Non-Winner State
↓
Winner Claims Prize
↓
Next Giveaway

The entire experience should feel **exciting but trustworthy**.

The user should feel motivated to participate without feeling that the platform is encouraging gambling.

---

78. Creative Freedom

You have creative freedom to improve the experience.

You may add:

- Reward reveal animations
- Prize detail drawer
- Entry progress
- "Your chance" indicator
- Personalized participation card
- Giveaway timeline
- Winner spotlight
- Reward vault
- Next giveaway preview
- Prize comparison
- Subtle 3D effects
- Interactive illustrations

However:

Every additional feature must improve the user experience and maintain the premium/trustworthy VELOOP Rewards identity.

Do not add features simply to make the page visually busy.

---

79. Individual Giveaway Details Page

Every **"Join Now"** button on a giveaway/prize card must open a **dedicated individual giveaway details page**.

For example:

```
/giveaway/iphone-15-pro
/giveaway/apple-watch
/giveaway/airpods
/giveaway/amazon-2000
/giveaway/amazon-500
/giveaway/amazon-20
```

The exact routing structure is up to you, but every giveaway must have its own identifiable details page.

---

80. Purpose of the Individual Page

The individual giveaway page should allow the user to understand everything before joining.

The user should clearly know:

- What they can win
- What the prize is
- How the giveaway works
- How much it costs to join
- What currency is required
- Giveaway duration
- Eligibility
- Terms & Conditions
- Winner selection process
- Prize claim process
- Important restrictions
- What happens after joining

The user should **never be surprised by the joining cost**.

---

81. Individual Giveaway Page Header

Create a premium header/hero section for every giveaway.

It should contain:

Prize Image

Use the provided assets where applicable.

For example:

- iPhone image
- Apple Watch image
- AirPods image
- Amazon Gift Card image

Giveaway Badge

Example:

```
 EXCLUSIVE GIVEAWAY
```

Prize Name

Example:

Win an iPhone 15 Pro

Short Description

Example:

Join this exclusive giveaway for a chance to win an iPhone 15 Pro.

Giveaway Status

- GIVEAWAY LIVE

Countdown

```
Ends in
12d : 08h : 45m
```

---

82. Individual Page Navigation

The page must have a navbar/header.

The user must always have an easy way to return to the main Giveaway page.

Example:

```
← Giveaway Home
```

or:

```
VELoop Rewards                    Giveaway Home
```

The back navigation should be clearly visible.

On mobile, use:

```
← Giveaway
```

---

83. Header Visuals

The individual page header should contain relevant imagery/illustrations.

For example:

iPhone Giveaway

Use the iPhone asset with subtle premium visual treatment.

Watch Giveaway

Use the Apple Watch asset.

AirPods Giveaway

Use the AirPods asset.

Amazon Gift Card

Use the gift-card asset.

You can additionally use:

- Subtle floating coins
- Reward particles
- Premium background shapes
- Soft lighting
- Abstract reward illustrations

But **do not make it excessively flashy**.

---

84. Prize Information Card

Below the hero, create a dedicated prize information area.

Example:

```
iPhone 15 Pro

Latest iPhone 15 Pro

 Winners: 1
 Participants: 2.3K+
 Ends in: 12d

Entry Fee
 250 VEs

[ Join Giveaway – 250 VEs ]
```

---

85. Giveaway Joining Fees

The following joining requirements must be represented accurately in the UI.

Giveaway	Joining Requirement
iPhone	**250 VEs**
Apple Watch	**200 VEs**
AirPods	**500 SVEs**
₹2,000 Amazon Voucher	**500 VEs**
₹500 Amazon Voucher	**300 VEs**

Giveaway	Joining Requirement
₹20 Voucher	**2,000 Tokens**

These values should be stored in the mock giveaway data rather than being hardcoded directly inside individual components.

86. Make the Cost Extremely Clear

Do not simply write:

Join Now

The user should understand the actual cost before clicking the final join button.

For example:

iPhone

```
Entry Fee

250 VEs

≈ Required balance: 250 VEs

[ Join for 250 VEs ]
```

For AirPods:

```
Entry Fee

500 SVEs

[ Join for 500 SVEs ]
```

For the ₹20 voucher:

```
Entry Fee

2,000 Tokens

[ Join for 2,000 Tokens ]
```

---

87. Balance Verification

The frontend should simulate checking the user's available balance.

Example:

```
Your Balance
```

```
350 VEs

Entry Fee
250 VEs

✓ You have enough VEs
```

Then:

```
[ Join Giveaway – 250 VEs ]
```

88. Insufficient Balance State

If the user doesn't have enough of the required currency:

Example:

```
Your Balance
120 VEs

Entry Fee
250 VEs

 Insufficient VEs

You need 130 more VEs to participate.
```

The primary CTA should change to:

Earn More VEs

instead of allowing the user to join.

For example:

```
[ Earn More VEs → ]
```

---

89. Currency-Specific Validation

The validation must use the correct currency.

iPhone

Check:

```
VEs >= 250
```

Watch

Check:

```
VEs >= 200
```

AirPods

Check:

```
SVEs >= 500
```

₹2,000 Amazon

Check:

```
VEs >= 500
```

₹500 Amazon

Check:

```
VEs >= 300
```

₹20 Voucher

Check:

```
Tokens >= 2000
```

Use mock data for the frontend.

---

90. How This Giveaway Works

Every individual giveaway page should contain a section:

How This Giveaway Works

Example:

```
01
Review the giveaway

↓

02
Check your eligibility

↓

03
Pay the required entry amount

↓

04
Your participation is recorded
```

```
↓

05
Wait until the giveaway ends

↓

06
Winner is selected

↓

07
Winner claims the prize
```

Make this visually engaging using icons and a timeline.

---

91. Important Terms & Conditions Section

Each individual giveaway page must contain a **Terms & Conditions** section.

Include relevant information such as:

Eligibility

Who can participate.

Entry Requirement

The exact VEs/SVEs/Tokens required.

Participation

How joining works.

Giveaway Duration

Start and end time.

Winner Selection

How winners are selected.

Winner Announcement

When winners are announced.

Prize Claim

How winners claim their prize.

Claim Deadline

How long winners have to submit their details.

Disqualification

Suspicious, fraudulent, abusive or rule-breaking activity may result in disqualification according to platform rules.

Refund/Entry Policy

Clearly explain what happens to the joining amount after participation.

Do not invent a refund policy. If the actual VELOOP policy is not finalized, mark the text as placeholder content for confirmation.

---

92. Important Information

Add an expandable section:

Important Information

Example:

Please review the giveaway rules and participation requirements carefully before joining.

Include:

- Entry currency
- Entry amount
- Giveaway duration
- Number of winners
- Prize details
- Winner selection
- Claim requirements
- Account eligibility
- Fraud prevention
- Platform rules

---

93. Prize Details

Create a dedicated section:

About the Prize

For example:

iPhone 15 Pro

Show:

- Product image
- Product name
- Short description
- Prize value if officially defined
- Number of winners
- Delivery/claim information

The same structure should work for all giveaway types.

---

94. Entry Fee Confirmation Modal

When the user clicks:

Join Giveaway – 250 VEs

do **not immediately join**.

Open a confirmation modal.

Example:

```
┌──────────────────────────────────┐
│        Confirm Participation     │
│                                  │
│  iPhone 15 Pro                   │
│                                  │
│ Entry Fee                        │
│ 250 VEs                          │
│                                  │
│ Your Balance                     │
│ 850 VEs                          │
│                                  │
│ Balance After Joining            │
│ 600 VEs                          │
│                                  │
│ By continuing, you confirm that  │
│ you have reviewed the giveaway   │
│ rules and terms.                 │
│                                  │
│ [ Cancel ]  [ Confirm & Join ]   │
└──────────────────────────────────┘
```

---

95. Confirmation for Other Currencies

For AirPods:

```
Entry Fee
500 SVEs

Your Balance
1,200 SVEs

After Joining
700 SVEs
```

For ₹20 voucher:

```
Entry Fee
2,000 Tokens
```

The modal should automatically display the appropriate currency.

---

96. Successful Join State

After confirmation, show a premium success state.

Example:

```
 You're In!

Your participation for the
iPhone 15 Pro giveaway
has been successfully recorded.

Entry Fee:
250 VEs

Good luck! 

[ View Giveaway ]
```

You may use a subtle success animation.

---

97. Already Joined State

If the user has already joined the giveaway, the page should not allow another duplicate entry unless the actual business rules explicitly allow multiple entries.

Show:

```
✓ You're Already Participating
```

```
Your entry has already been recorded.

[ View Giveaway Status ]
```

98. Login Requirement

If a visitor is not logged in and clicks the join button:

```
Login Required

Please login to your VELOOP Rewards account
before participating in this giveaway.

[ Login ]
[ Create Account ]
And redirect to the /login page
```

Do not allow unauthenticated participation.

---

99. Giveaway Participation Rules

The individual page should clearly explain whether the giveaway permits:

- One participation per user
- Multiple entries
- Re-entry
- Additional entries through tasks

If these rules are not finalized, use clearly marked mock/demo wording and structure the UI so the real rules can later be inserted.

---

100. Responsive Individual Page

The individual giveaway page must be fully responsive.

Mobile – 320px+

Hero:

```
Image
↓
Prize Name
↓
Countdown
↓
Entry Fee
```

```
↓
CTA
```

Tablet

Use a two-column hero where space allows.

Desktop

Use:

```
Prize Visual          Giveaway Information
```

with a premium composition.

---

101. Individual Page Footer

Include a compact footer with:

- Giveaway Home
- Rules
- Terms
- Privacy
- Support
- VELOOP Rewards branding

Also include:

Have questions? Contact VELOOP Rewards support.

---

102. Updated Giveaway User Journey

The complete experience should now be:

```
Giveaway Home
       ↓
User sees Prize
       ↓
Clicks "Join Now"
       ↓
Individual Giveaway Page
       ↓
Reviews Prize
       ↓
Checks Entry Fee
       ↓
Reads Rules & Terms
       ↓
Checks Eligibility
```

```
        ↓
Clicks Join
        ↓
Balance Verification
        ↓
Confirmation Modal
        ↓
Confirm & Join
        ↓
Participation Successful
        ↓
Return to Giveaway
        ↓
Wait for Giveaway End
        ↓
Winner Announcement
        ↓
Winner / Non-Winner
        ↓
Winner Claims Prize
```

---

103. Final Mandatory Requirement

Every "Join Now" button on the main Giveaway page must lead to the corresponding individual giveaway details page.

It must **not** directly join the giveaway.

The individual page is the place where the user:

1. Understands the prize
2. Reviews the entry fee
3. Reads the terms
4. Checks eligibility
5. Checks their balance
6. Confirms participation
7. Pays the required virtual currency
8. Receives confirmation

This flow should be implemented with clean React state and reusable components so the same system works for **iPhone, Watch, AirPods, Amazon vouchers and future giveaways**.

Final Instruction

Build the **Giveaway Banner + Complete Giveaway Page** as a polished, production-quality frontend experience.

Use the provided images and assets as visual references, but create your own VELOOP-branded implementation.

The final page must be:

Premium + Rewarding + Interactive + Trustworthy + Responsive + Engaging + Realistic

and should be designed so that the frontend can later be connected directly to a real giveaway backend without requiring a complete rewrite of the UI architecture.

BACKEND

VELOOP Rewards – Giveaway Backend Development & Fraud Protection Task

1. Task Objective

The existing Giveaway frontend must be connected to a complete, secure and scalable backend.

The backend must become the **single source of truth** for:

- Giveaway events
- Giveaway status
- Prize information
- Entry fees
- VEs
- SVEs
- Tokens
- User participation
- Participation status
- Winner records
- Previous winners
- Entry transactions
- Prize claims
- Eligibility
- Fraud detection
- Suspicious participation
- Audit logs

Critical Principle

Never trust values coming from the frontend.

The frontend may request:

"I want to join Giveaway X."

But the backend must independently verify:

- Is the giveaway active?
- Does the giveaway exist?
- Is the user logged in?
- Is the user eligible?
- Has the user already participated?
- Does the user have sufficient balance?
- Is the required currency correct?
- Is the entry fee correct?
- Is the request duplicated?
- Is the account suspicious?
- Is the participation allowed?

Only then should the backend process the participation.

---

2. Technology Stack

Backend

- Node.js
- Express.js
- MongoDB
- Mongoose

Authentication

Use the existing VELOOP authentication system where possible.

The backend should support secure authentication through:

- JWT/access tokens
- Secure authentication middleware
- Refresh-token strategy if already used by the project

Additional Libraries

You may use suitable libraries for:

- Validation
- Rate limiting
- Security headers
- Password/authentication handling
- Logging
- Request sanitization
- Device/IP analysis
- Unique ID generation

3. Backend Architecture

Suggested structure:

```
backend/
│
├── src/
│   │
│   ├── controllers/
│   │   ├── giveawayController.js
│   │   ├── participationController.js
│   │   ├── winnerController.js
│   │   ├── claimController.js
│   │   └── adminGiveawayController.js
│   │
│   ├── models/
│   │   ├── Giveaway.js
│   │   ├── Prize.js
│   │   ├── GiveawayParticipation.js
│   │   ├── GiveawayEntryTransaction.js
│   │   ├── GiveawayWinner.js
│   │   ├── PrizeClaim.js
│   │   ├── FraudEvent.js
│   │   └── AuditLog.js
│   │
│   ├── routes/
│   │   ├── giveawayRoutes.js
│   │   ├── participationRoutes.js
│   │   ├── winnerRoutes.js
│   │   └── claimRoutes.js
│   │
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   ├── rateLimitMiddleware.js
│   │   ├── validationMiddleware.js
│   │   ├── fraudMiddleware.js
│   │   └── errorMiddleware.js
│   │
│   ├── services/
│   │   ├── giveawayService.js
│   │   ├── participationService.js
│   │   ├── balanceService.js
│   │   ├── fraudService.js
│   │   └── winnerService.js
│   │
│   ├── utils/
│   ├── validators/
│   ├── config/
│   └── app.js
│
└── server.js
```

The exact architecture can differ, but the code should remain modular.

---

4. Giveaway Must Be Stored in Database

Do **not** hardcode giveaway information in React.

The database should contain the complete giveaway configuration.

Example:

```
Giveaway
│
├── id
├── title
├── slug
├── description
├── status
├── startAt
├── endAt
├── rules
├── eligibility
├── prizes
├── participationSettings
└── timestamps
```

---

5. Giveaway Status

The backend should control the giveaway lifecycle.

Possible statuses:

```
UPCOMING
ACTIVE
ENDED
ARCHIVED
```

The frontend must receive this status from the backend.

Do not allow the frontend to decide:

```
if countdown === 0
   giveawayEnded = true
```

The frontend may visually display the countdown, but the backend remains authoritative.

---

6. Giveaway Lifecycle

Example:

UPCOMING
↓
ACTIVE
↓
END TIME REACHED
↓
ENDED
↓
WINNERS SELECTED
↓
ARCHIVED

After the event ends:

- New participation must be rejected.
- Existing participation remains recorded.
- Winners can be selected.
- Winner claims can be processed.
- The event moves to history.
- A new giveaway can become active.

---

7. One Participation Per User

Mandatory Requirement

A user can participate **only once in a particular giveaway event**.

For example:

Giveaway: Summer iPhone Giveaway
User: VE10025

Participation:
YES

If the same user tries again:

Participation already exists.

The backend must reject the request.

---

8. Database-Level Protection

Do not rely only on:

```
if (alreadyJoined) return;
```

because two simultaneous requests can bypass a simple application-level check.

Create a **compound unique database index** such as:

```
userId + giveawayId
```

This guarantees that one user cannot have multiple participation records for the same giveaway.

Example concept:

```
unique(userId, giveawayId)
```

This is extremely important.

---

9. New Giveaway Participation

The restriction applies **per event**, not permanently.

Example:

```
Giveaway A
User participated ✓

Giveaway A ended

Giveaway B started

User can participate in Giveaway B ✓
```

The user cannot participate twice in Giveaway A, but can participate once in Giveaway B.

---

10. Entry Fee Must Come From Backend

The frontend must **never send the actual price as a trusted value**.

Bad approach:

```
POST /join

{
  giveawayId: "...",
  amount: 250
}
```

The backend should determine the fee from the giveaway database.

For example:

```
giveawayId
    ↓
Database
    ↓
Prize configuration
    ↓
250 VEs
```

If a malicious user changes the frontend request to:

```
amount = 1
```

the backend must ignore it.

---

11. Giveaway Entry Configuration

Store:

Prize	Currency	Entry Fee
iPhone	VEs	250
Apple Watch	VEs	200
AirPods	SVEs	500
Amazon ₹2,000	VEs	500
Amazon ₹500	VEs	300
Amazon ₹20	Tokens	2,000

The backend must read these values from the database.

---

12. Balance Verification

Before participation:

```
User Balance
    ↓
Required Currency
    ↓
Entry Fee
    ↓
Compare
```

Example:

User VEs = 350

Entry Fee = 250 VEs

350 >= 250

Eligible ✓

If:

User VEs = 100

Entry Fee = 250

100 < 250

Reject

---

13. Correct Currency Validation

The backend must ensure the correct balance is checked.

For example:

iPhone

VEs

AirPods

SVEs

₹20 Voucher

Tokens

A user must never be able to substitute:

Tokens → VEs

just by changing the request.

---

14. Atomic Balance Deduction

Participation and balance deduction must be handled safely.

The system should avoid:

Check balance
↓
Deduct later

without transaction protection.

Use a MongoDB transaction/session where appropriate so that:

Participation Created
+
Balance Deducted
+
Transaction Recorded

either all succeed or all fail.

This prevents situations such as:

User is charged but participation isn't created.

or:

Participation is created but the user isn't charged.

---

15. Entry Transaction Record

Every successful participation should generate a transaction record.

Example:

GiveawayEntryTransaction

userId
giveawayId
prizeId
currency
amount
type
status
balanceBefore
balanceAfter
transactionId
createdAt

Example:

250 VEs deducted

Before: 850 VEs
After: 600 VEs

---

16. Transaction Status

Possible statuses:

PENDING
SUCCESS
FAILED
REVERSED

This makes the system easier to audit.

---

17. Idempotency Protection

Protect the join endpoint against repeated requests.

Example:

A user rapidly clicks:

JOIN
JOIN
JOIN
JOIN

The backend must not deduct 250 VEs four times.

Use an idempotency/request key mechanism.

The same request should produce the same result rather than generating multiple participations.

---

18. Race Condition Protection

The system must handle simultaneous requests.

Example:

Request A → Join
Request B → Join

at almost exactly the same time.

The database must still produce:

1 Participation
1 Entry Fee Deduction

not:

2 Participations
2 Deductions

Database unique indexes + transaction handling should be used.

---

19. Giveaway End Protection

The backend must check the current server time.

Example:

startAt <= now <= endAt

Only then can the user participate.

If:

now > endAt

return:

GIVEAWAY_ENDED

Even if the frontend still shows an old "Join" button.

---

20. Frontend Cannot Override Giveaway Status

A malicious user must not be able to change:

status: ACTIVE

from browser developer tools.

The backend determines the real status.

---

21. Fraud Protection Architecture

Create a dedicated fraud-protection layer.

Potential signals include:

- User account
- Device identifier
- IP address
- Session information
- Request frequency
- Account age
- Participation history
- Suspicious account patterns
- Repeated failed attempts
- Multiple accounts using the same device signals
- Abnormal participation patterns

No single signal should automatically be treated as definitive proof of fraud.

---

22. Device Tracking

Generate a privacy-conscious device/session identifier for abuse prevention.

Example concept:

deviceHash

Store a hashed identifier rather than unnecessary raw device information.

The goal is:

Detect repeated participation attempts from the same environment.

Not:

Collect unnecessary personal information.

---

23. Same Device Protection

If the business rule is:

One device may participate only once in a particular giveaway.

then the backend should maintain a suitable participation/device record.

Example:

giveawayId
deviceHash
userId
status
createdAt

A second account attempting to participate from the same device should be flagged for review or rejected according to the platform's finalized policy.

---

24. Important Device Limitation

The intern must understand:

Device fingerprinting is an abuse-prevention signal, not an unbreakable identity system.

Users can:

- Change devices
- Clear browser data
- Use different browsers
- Use VPNs
- Modify fingerprint characteristics

Therefore, device detection must be combined with other protections.

---

25. Self-Participation / Multiple Account Detection

If the same person attempts to create another account and participate in the same giveaway, the system should evaluate multiple signals.

Potential signals:

Device
IP
Account behavior
Participation history
Session patterns
Other approved anti-abuse signals

Do not automatically ban legitimate users based solely on shared Wi-Fi/IP because multiple legitimate users may use the same network.

---

26. Suspicious Participation

Create a fraud risk system.

Example:

riskScore: 0–100

Possible classifications:

0–29 LOW
30–59 MEDIUM
60–79 HIGH
80–100 CRITICAL

The exact scoring model can be proposed by the intern and reviewed before production.

---

27. Fraud Events

Create a collection:

FraudEvent

Possible information:

userId
giveawayId
deviceHash
riskScore
reason
signals
action
createdAt

Example:

Reason:
Repeated participation attempt

Action:
BLOCKED

---

28. Do Not Reward Suspicious Participation

If a participation attempt is flagged as fraudulent:

Do NOT:
- Deduct reward incorrectly
- Create duplicate entry
- Generate winner eligibility
- Give additional entries

Instead:

FLAGGED / BLOCKED

depending on the severity and policy.

---

29. Audit Logging

Every important financial/reward action should be auditable.

Record:

- User
- Action
- Giveaway
- Amount
- Currency
- Result
- Timestamp
- Request identifier
- Relevant security information

Examples:

JOIN_GIVEAWAY
ENTRY_FEE_DEDUCTED
JOIN_REJECTED
DUPLICATE_ATTEMPT
FRAUD_FLAGGED
CLAIM_SUBMITTED
WINNER_SELECTED

---

30. Winner Selection

Winner selection must be backend-controlled.

Never select winners using frontend JavaScript.

The frontend should only receive:

winner information

after the backend has finalized it.

---

31. Winner Integrity

Once winners are selected:

- Store the winner record.
- Associate it with the giveaway.
- Associate it with the prize.
- Store selection timestamp.
- Prevent accidental duplicate winner records.

Example:

GiveawayWinner

giveawayId
prizeId
userId
selectionMethod
selectedAt
status

---

32. One iPhone Winner

For the iPhone prize:

winnerCount = 1

The backend must enforce this.

If three requests/processes attempt to assign an iPhone winner, only the configured number of winners should be created.

---

33. Multiple Winners

For prizes such as:

- Apple Watch
- AirPods
- Amazon vouchers

the number of winners should be configurable.

Example:

```
prize:
  winnerCount: 5
```

The backend should enforce the configured limit.

---

34. Winner Claim Protection

A winner should only be able to claim a prize if:

```
authenticatedUserId === winner.userId
```

The frontend must not be trusted for this.

A malicious user should not be able to change:

```
winnerId
```

in the request.

---

35. Prize Claim API

Example:

```
POST /api/giveaways/:giveawayId/claim
```

The backend should determine:

```
Authenticated User
      ↓
Winner?
      ↓
Which Prize?
      ↓
Prize Type?
      ↓
Required Information
```

---

36. Physical Prize Claim

For physical prizes:

- Name
- Phone
- Address
- City
- State
- PIN

should be validated server-side.

---

37. Gift Card Claim

For Amazon gift cards:

- Email address

should be validated.

The backend should determine the prize type instead of trusting:

```
type = amazon
```

from the frontend.

---

38. Sensitive Claim Information

Prize claim information should be treated as sensitive.

Do not:

- Return it unnecessarily through public APIs.
- Include it in winner lists.
- Expose it in frontend logs.
- Include it in URLs.
- Display it publicly.

Only authorized backend processes/admins should access it.

---

39. API Design

Suggested endpoints:

Giveaway

GET /api/giveaways/current
GET /api/giveaways/:id
GET /api/giveaways/previous

Participation

GET /api/giveaways/:id/my-status
POST /api/giveaways/:id/join

Winners

GET /api/giveaways/:id/winners
GET /api/giveaways/previous/winners

Claim

POST /api/giveaways/:id/claim
GET /api/giveaways/:id/my-claim

The exact API structure may be improved by the intern.

---

40. Example Join Request

Frontend sends only the necessary information:

```
{
  "giveawayId": "GW-2026-08"
}
```

The backend determines everything else.

The frontend should **not send**:

amount
currency
userId
balance
prize price
winner status

as trusted values.

41. Example Backend Flow

POST /join
↓
Authenticate User
↓
Validate Request
↓
Check Rate Limit
↓
Load Giveaway
↓
Check Giveaway Status
↓
Check Start/End Time
↓
Check Prize
↓
Check User Eligibility
↓
Check Existing Participation
↓
Check Fraud Signals
↓
Check Correct Balance
↓
Start Database Transaction
↓
Deduct Correct Currency
↓
Create Participation
↓
Create Transaction Record
↓
Commit
↓
Return Success

---

42. Error Responses

Create meaningful API errors.

Examples:

GIVEAWAY_NOT_FOUND
GIVEAWAY_NOT_ACTIVE
GIVEAWAY_ENDED
ALREADY_PARTICIPATING
INSUFFICIENT_VE_BALANCE
INSUFFICIENT_SVE_BALANCE
INSUFFICIENT_TOKEN_BALANCE

LOGIN_REQUIRED
PARTICIPATION_BLOCKED
SUSPICIOUS_ACTIVITY
RATE_LIMITED
CLAIM_NOT_ALLOWED

The frontend should convert these into friendly messages.

---

43. Example User Messages

Instead of:

MongoServerError: E11000 duplicate key error

show:

You're already participating □
You can participate again when a new giveaway event begins.

For insufficient balance:

Not enough VEs
You need 130 more VEs to join this giveaway.

For ended giveaway:

This giveaway has ended
Check out the winners and get ready for the next giveaway.

For suspicious activity:

Participation couldn't be completed
We couldn't verify this participation request. Please try again later or contact support if you believe this is an error.

Do not reveal detailed fraud-detection logic to the user.

---

44. Rate Limiting

Protect sensitive endpoints such as:

POST /join
POST /claim
POST /login

from excessive requests.

The intern should implement appropriate rate limits and return a friendly response when the limit is exceeded.

---

45. Security Middleware

Use appropriate Express security practices, including:

- Security headers
- CORS configuration
- Request validation
- Payload limits
- Rate limiting
- Authentication middleware
- Authorization middleware
- Input sanitization
- Centralized error handling

Never use:

```
cors(*)
```

in production without understanding the implications.

---

46. Environment Variables

Sensitive configuration must never be committed to GitHub.

Use:

```
.env
```

for:

```
MONGO_URI
JWT_SECRET
REFRESH_SECRET
CLIENT_URL
```

and any other sensitive configuration.

Commit:

```
.env.example
```

instead.

---

47. No Secrets in Frontend

Never place:

- MongoDB credentials
- JWT signing secrets
- Admin credentials
- Private API keys

inside React/Vite environment variables that are exposed to the browser.

---

48. Authorization

Separate:

User

Can:

- View giveaway
- Participate
- View own participation
- View public winners
- Claim own prize

Admin

Can, depending on finalized permissions:

- Create giveaway
- Update giveaway
- Configure prizes
- Start/end event
- View participation
- Select/finalize winners
- Process claims

---

49. Admin Actions Must Be Protected

Never create an endpoint where simply knowing:

/admin/select-winner

allows anyone to select winners.

Admin endpoints require:

Authentication
+
Authorization
+
Validation

---

50. Frontend Data Loading

The frontend must fetch giveaway data from the backend.

Example:

GET /api/giveaways/current

The UI should then display:

- Giveaway title
- Status
- Countdown
- Prize
- Entry fee
- Participants
- Winners
- User participation status

---

51. Themed Giveaway Loader

Mandatory Requirement

While the frontend is fetching giveaway information, **do not display a generic Bootstrap spinner**.

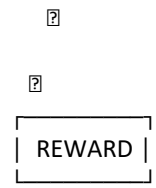
Create a completely custom VELOOP Giveaway-themed loader.

The loader should feel like part of the giveaway experience.

---

52. Giveaway Loader Concept

For example:



Unlocking rewards...

• • • • •

The exact design is up to the developer.

---

53. Loader Animation Ideas

You can create:

- Gift box opening animation
- Reward particles
- Rotating ticket
- Coin movement
- Prize card reveal
- Trophy pulse
- Loading progress ring
- Subtle sparkle animation

Keep the animation premium.

---

54. Loader Text

Rotate between messages such as:

Preparing today's rewards...

Checking active giveaways...

Loading available prizes...

Bringing your rewards closer...

Do not make the messages change too quickly.

---

55. Loader States

Create different loaders where appropriate:

Initial Page Load

Full-page giveaway loader.

Prize Data Loading

Prize-card skeleton.

Winner Data Loading

Winner-section skeleton.

Joining

Button-level loading state:

```
Joining...
```

Claim Submission

```
Submitting claim...
```

---

56. Joining Loader

When the user confirms:

```
Confirm & Join
```

the button should immediately enter a loading state:

```
⟳ Joining Giveaway...
```

The button should be disabled during the request.

This prevents accidental repeated clicks.

---

57. Success Animation

After successful participation:

You're In!

Your entry has been successfully recorded.

Use a subtle reward animation.

---

58. Failed Participation

If the API returns an error:

- Stop loader.
- Re-enable button where appropriate.
- Display a clear error.
- Do not falsely show success.

---

59. Backend-Driven Loader

The frontend should have proper states:

loading
success
error
empty

Example:

Loading
↓
API Response
↓
Success → Render Giveaway

or

Error → Show Retry

or

No Active Giveaway → Show Empty State

---

60. No Fake Frontend Data After API Integration

Once the backend is connected, the page should not silently fall back to fake giveaway statistics.

If the API fails:

Unable to load giveaway

instead of showing fake participants/winners as if they were real.

---

61. Database Models

At minimum, create models for:

Giveaway

Giveaway

Prize

GiveawayPrize

Participation

GiveawayParticipation

Entry Transaction

GiveawayEntryTransaction

Winner

GiveawayWinner

Claim

PrizeClaim

Fraud Event

FraudEvent

Audit Log

AuditLog

---

62. Participation Model

Suggested fields:

userId
giveawayId
prizeId
entryCurrency
entryAmount
deviceHash
status
joinedAt
transactionId

Create a unique index:

userId + giveawayId

---

63. Important Business Rule

The system should enforce:

One user = one participation per giveaway event.

Once the giveaway ends:

Participation locked

When the next giveaway starts:

New participation record

is allowed.

---

64. Previous Winners

When a new event begins, previous winners do not need to be physically moved to another collection.

Instead, winners remain associated with their original giveaway.

The frontend can query:

GET /giveaways/previous

and display completed-event winners.

This preserves historical integrity.

---

65. Giveaway History

Every giveaway should remain permanently identifiable.

Example:

GW-001
Summer Giveaway

GW-002
Independence Rewards

GW-003
September Rewards

Do not overwrite old giveaway records.

---

66. Balance Integrity

The backend should use the user's authoritative wallet/balance record.

Do not calculate:

VEs = frontend localStorage

The frontend balance is only for display.

The backend determines the actual available balance.

---

67. Transaction History

Every entry fee deduction should appear in the appropriate transaction history.

Example:

Giveaway Entry

-250 VEs

Summer iPhone Giveaway

19 Aug 2026

This makes the system transparent and easier to audit.

---

68. Refund/Reversal Architecture

Even if refunds are not currently supported, structure the transaction model so that a reversal can be recorded later if the business policy requires it.

Do not simply delete the transaction.

Use a compensating transaction such as:

REVERSAL

This preserves the financial audit trail.

---

69. Testing Requirements

The intern must test:

Normal

- ☐ User joins successfully
- ☐ Correct currency deducted
- ☐ Correct amount deducted
- ☐ Participation created
- ☐ Transaction created

Duplicate

- ☐ User clicks twice
- ☐ User sends two requests simultaneously
- ☐ Only one participation exists
- ☐ Only one deduction occurs

Balance

- [ ] Sufficient balance
- [ ] Insufficient balance
- [ ] Wrong currency attempt

Giveaway

- [ ] Upcoming giveaway
- [ ] Active giveaway
- [ ] Ended giveaway
- [ ] Archived giveaway

Security

- [ ] Unauthenticated join
- [ ] Modified user ID
- [ ] Modified amount
- [ ] Modified currency
- [ ] Modified prize ID
- [ ] Replayed request
- [ ] Excessive requests
- [ ] Suspicious device activity

---

70. Fraud Testing Examples

Test malicious scenarios such as:

User has 100 VEs

Frontend request:
amount = 1

Expected:
REJECT / Ignore amount

Another:

User already participated

POST /join again

Expected:
ALREADY_PARTICIPATING

Another:

Giveaway ended

POST /join

Expected:
GIVEAWAY_ENDED

Another:

User A changes request userId → User B

Expected:
Backend ignores client-provided identity

The authenticated identity must come from the secure authentication context.

---

71. Deliverables

The intern must submit:

Backend

Complete Node/Express/MongoDB backend.

Frontend

Connected giveaway frontend.

Database

All giveaway-related collections/models.

Security

Authentication, authorization, validation and fraud-protection middleware.

Documentation

Complete README.md.

API Documentation

Document:

- Endpoint
- Method
- Authentication
- Request
- Response
- Error codes

GitHub

Push the complete project to GitHub.

Deployment

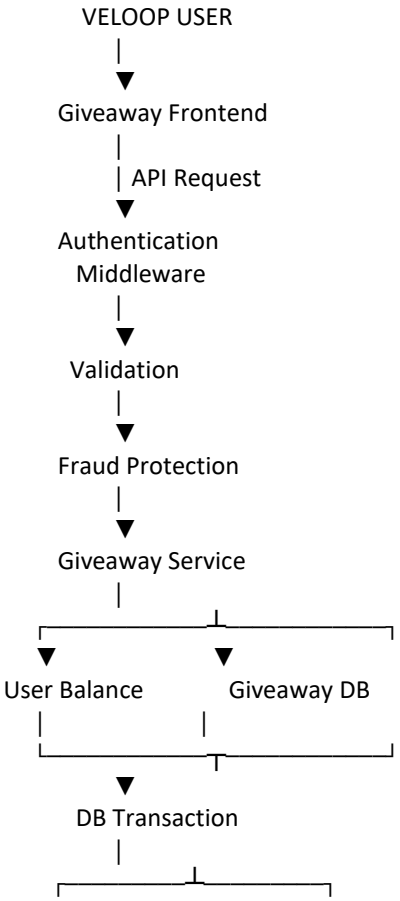
Deploy the frontend and backend appropriately.

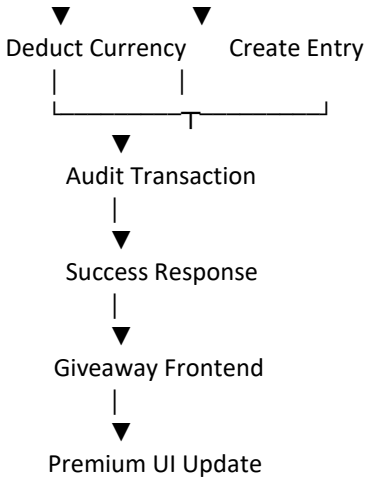
If the backend is deployed separately, provide the backend API URL.

---

72. Final Architecture

The final system should work approximately like this:





---

73. Most Important Security Rule

The intern must understand this clearly:

Anything visible or editable in the browser must be considered untrusted.

The user can modify:

- JavaScript
- API requests
- Request body
- Local storage
- React state
- Displayed balance
- Prize ID
- Entry fee
- Currency
- User ID

Therefore, the backend must independently determine all critical values.

---

74. Final Expected Experience

The finished Giveaway system should provide this experience:

```
User opens Giveaway
      ↓
Custom Giveaway Loader
      ↓
Backend data loads
      ↓
Current giveaway displayed
```

↓
User opens prize
↓
Individual Giveaway Page
↓
Reads prize + rules + T&C
↓
Checks entry fee
↓
Backend checks balance
↓
User confirms
↓
Fraud/security checks
↓
Atomic transaction
↓
Currency deducted
↓
Participation recorded
↓
Success animation
↓
User participates
↓
Giveaway ends
↓
Backend finalizes winners
↓
Winner list appears
↓
Winner gets claim option
↓
Prize claim submitted

Final Quality Standard

This should **not** be treated as a simple CRUD backend.

The goal is to build the foundation of a real reward/giveaway system where:

Frontend = presentation

Backend = authority

Database = source of truth

Transactions = financial/reward integrity

Fraud layer = abuse prevention

Audit logs = accountability

Custom loader = premium user experience

And the complete implementation should be structured so that the giveaway system can later support **multiple giveaway events, multiple prizes, different currencies, different entry fees, winner limits, future reward types, and additional anti-abuse rules** without rebuilding the architecture.

Project Timeline Submission Deadline

To ensure smooth project execution and timely review:

Project Assigned On:	**Today when its assigned**
Duration:	**20 – 24 Days from its assigned date**
Submission Tine:	**5:30 P.M. (IST)**